

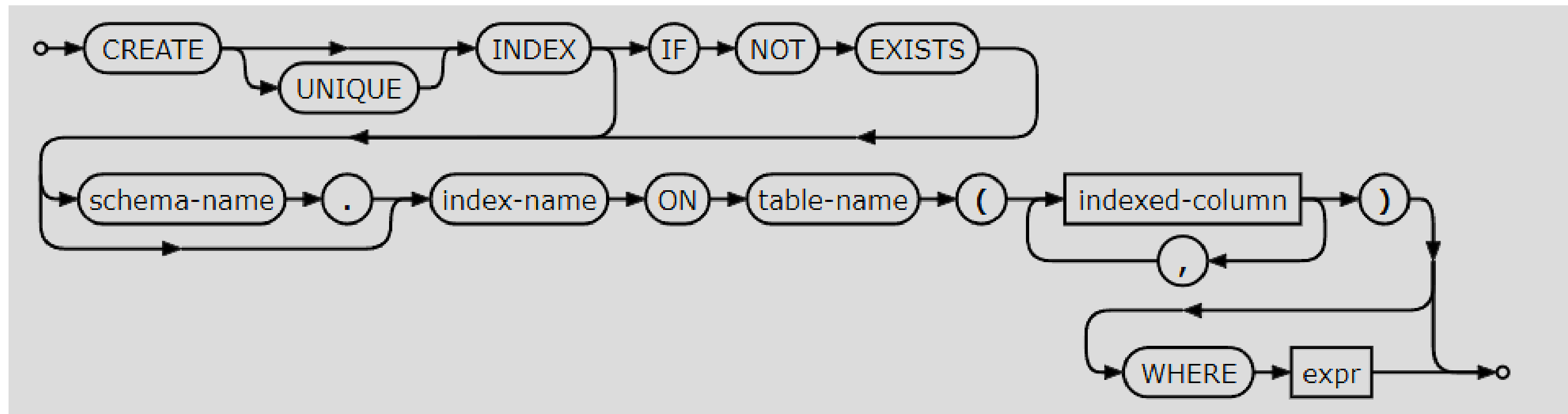


Ayudantía 9

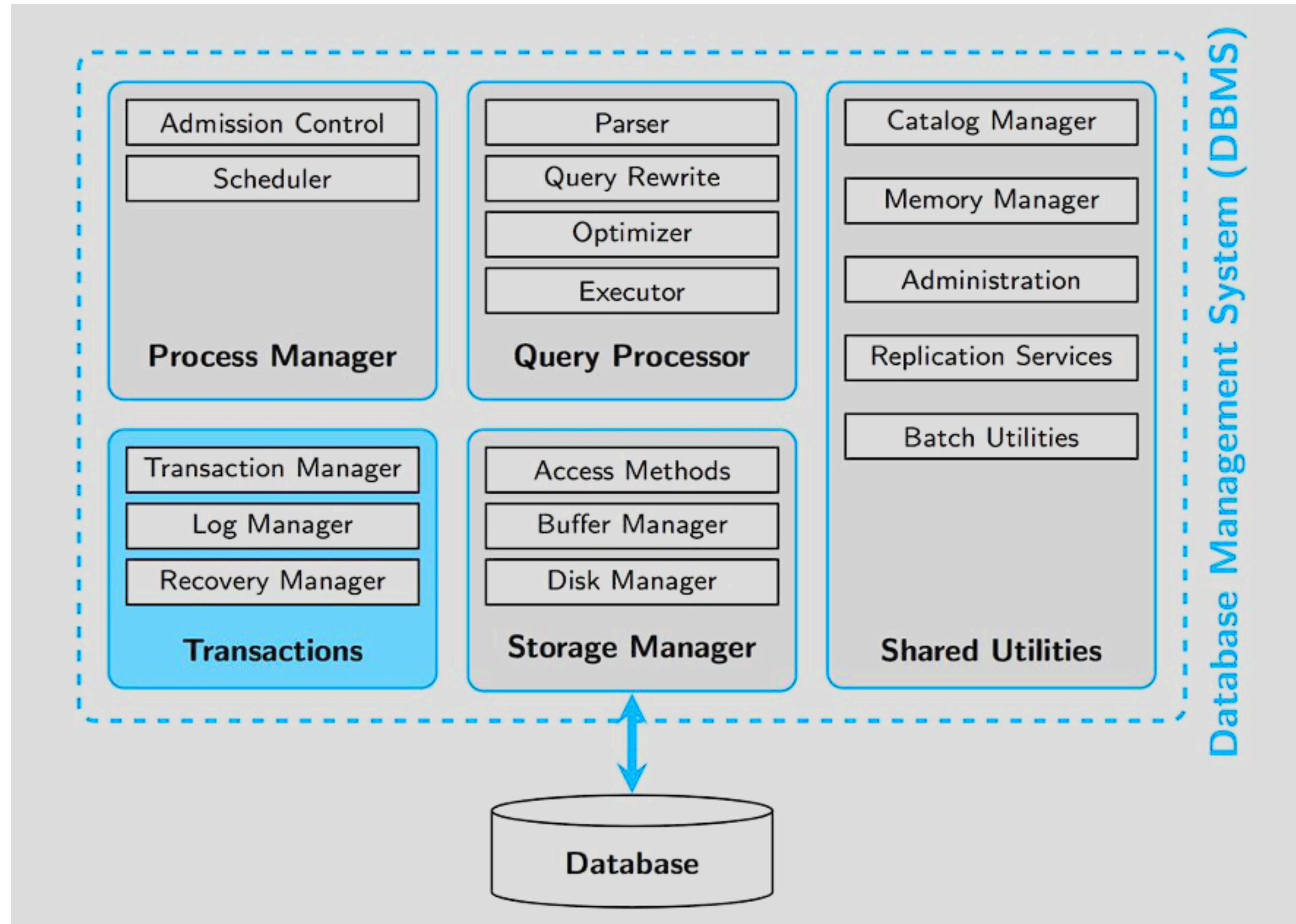
Profesor: Alex DI Genova B.
Ayudante: Enzo Vásquez C.

Recordatorio

- **Índices:** Estructura de datos que optimiza la velocidad de búsqueda en una tabla. Funciona exactamente igual que el índice de un libro: en lugar de leer registro por registro, el motor de base de datos utiliza el índice como atajo para ubicar la información de forma ultrarrápida.



DBMS



Transacciones

Transacciones

Una **transacción** es una secuencia de 1 o más **operaciones** que modifican o consultan la base de datos.

- Transferencias de dinero entre cuentas
- Compra por internet
- Inscribir un curso

El principal propósito de las transacciones es garantizar la **integridad** de los datos y la **consistencia** del estado de la base de datos, incluso en presencia de fallos y errores.

Ejemplo motivador

Supongamos que la Julia y el Pepe tienen una cuenta bancaria en común (con 10000 inicialmente) y cada uno le va a transferir al Cristian (con 6000 inicialmente).

- La Julia quiere transferirle 5000 a su amigo Cristian.
- El Pepe quiere transferirle 2500 a su amigo Cristian.

El proceso:

Proceso Julia	Proceso Pepe	Saldo Cuenta J & P	Saldo Cuenta Cristian
READ(saldoJP, x)		10.000	
WRITE(saldoJP, x - 5000)		5.000	
READ(saldoC, x)			6.000
WRITE(saldoC, x + 5000)			11.000
	READ(saldoJP, y)	5.000	
	WRITE(saldoJP, y - 2500)	2.500	
	READ(saldoC, y)		11.000
	WRITE(saldoC, y + 2500)		13.500

Pero el acceso es **concurrente**, opción 1:

Proceso Julia	Proceso Pepe	Saldo Cuenta J & P	Saldo Cuenta Cristian
READ(saldoJP, x)		10.000	
WRITE(saldoJP, x - 5000)		5.000	
	READ(saldoJP, y)	5.000	
	WRITE(saldoJP, y - 2500)	2.500	
	READ(saldoC, y)		6.000
	WRITE(saldoC, y + 2500)		8.500
READ(saldoC, x)			8.500
WRITE(saldoC, x + 5000)			13.500

Pero el acceso es **concurrente**, opción 2:

Proceso Julia	Proceso Pepe	Saldo Cuenta J & P	Saldo Cuenta Cristian
READ(saldoJP, x)		10.000	
WRITE(saldoJP, x - 5000)		5.000	
READ(saldoC, x)			6.000
	READ(saldoJP, y)	5.000	
	WRITE(saldoJP, y - 2500)	2.500	
	READ(saldoC, y)		6.000
	WRITE(saldoC, y + 2500)		8.500
WRITE(saldoC, x + 5000)			11.000

Se perdieron 2500!
¿Por qué?

¿Qué pasó?

Mezclamos las operaciones a realizar (en cada depósito).

- **El Ideal:** cada depósito se ejecuta en el orden que fue solicitado
- **Lo real:** Para optimizar accesos a disco, nos conviene mezclar operaciones.

El DBMS mezcla las operaciones para optimizar la ejecución y al mismo tiempo mantiene las transacciones.

Una solución sería agrupar las operaciones de modo que se ejecuten como una sola, se hace completa o no se hace.

Proceso Julia	Proceso Pepe	Saldo Cuenta J & P	Saldo Cuenta Cristian
Comienzo T1		10.000	6.000
READ(saldoJP, x)			
WRITE(saldoJP, x - 5000)			
READ(saldoC, x)			
WRITE(saldoC, x + 5000)			
Fin T1		5000	11000
	Comienzo T2	5000	11000
	READ(saldoJP, y)		
	WRITE(saldoJP, y - 2500)		
	READ(saldoC, y)		
	WRITE(saldoC, y + 2500)		
	Fin T2	2500	13500

Pero para optimizar el tiempo, todas las transacciones son concurrentes!

Proceso Julia	Proceso Pepe	Saldo Cuenta J & P	Saldo Cuenta Cristian
Comienzo T1	Comienzo T2	10.000	6.000
READ(saldoJP, x)	READ(saldoJP, y)		
WRITE(saldoJP, x - 5000)	WRITE(saldoJP, y - 2500)		
READ(saldoC, x)	READ(saldoC, y)		
WRITE(saldoC, x + 5000)	WRITE(saldoC, y + 2500)		
Fin T1	Fin T2	2500	13500

Entonces qué hacemos?

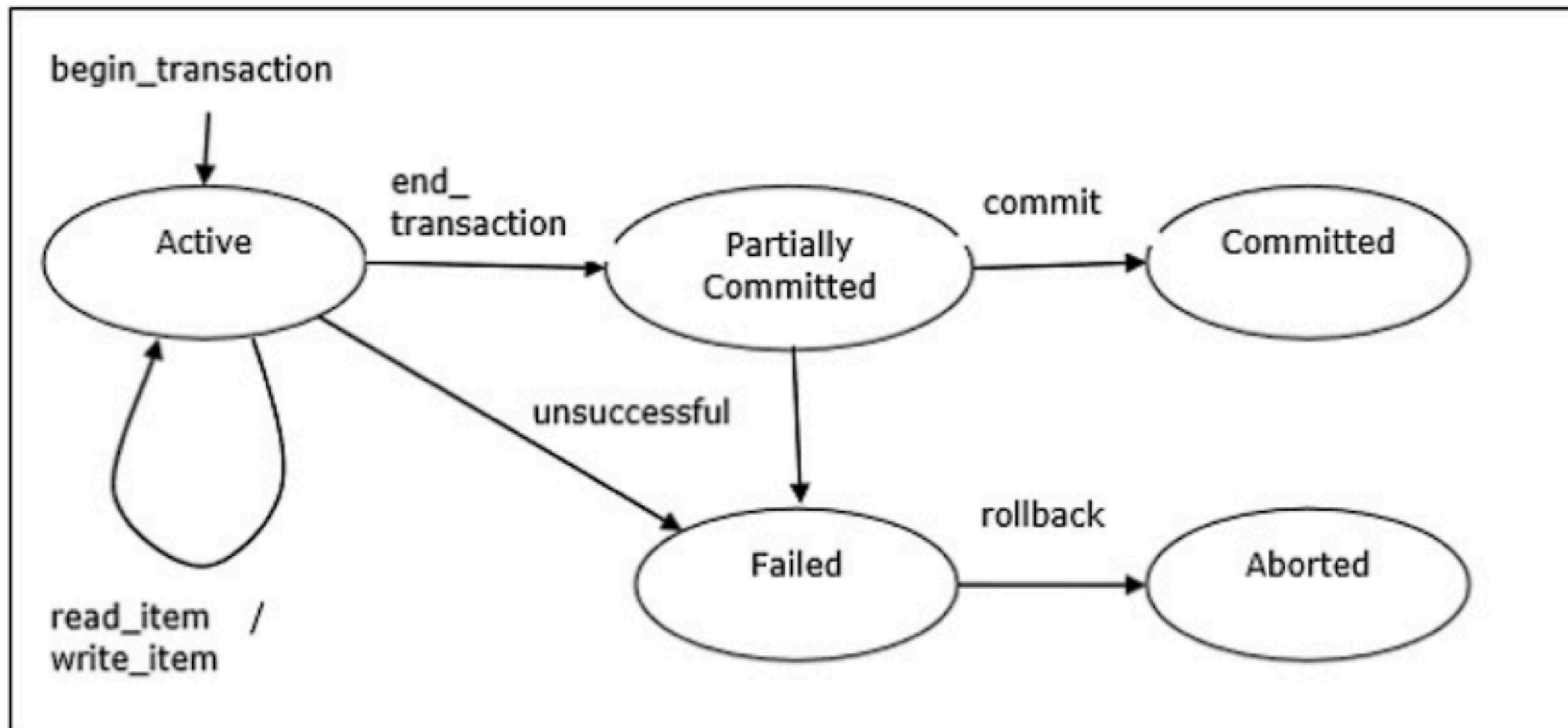
ACID 

ACID

- **Atomicity:** O se ejecutan todas las operaciones de la transacción, o no se ejecuta ninguna.
- **Consistency:** Cada transacción preserva la consistencia de la BD (restricciones de integridad, etc.).
- **Isolation:** Cada transacción debe ejecutarse como si se estuviese ejecutando sola, de forma aislada.
- **Durability:** Los cambios que hace cada transacción son permanentes en el tiempo, independiente de cualquier tipo de falla.

Operaciones básicas

- **READ(X), R(X)**: Lectura del elemento X.
- **WRITE(X), W(X)**: Escritura del elemento X.
- **ABORT, A**: Abortar la transacción.
- **COMMIT, C**: Finalizar la transacción.



Conflictos con transacciones

- **Lecturas sucias** (Write - Read): ocurren cuando una transacción accede a datos que han sido modificados por otra transacción que aún no se ha terminado.
- **Lecturas no repetibles** (Read - Write): ocurren cuando una fila es leída dos veces y el valor cambia entre ambas lecturas. Esto se debe a que otra transacción actualiza o modifica la fila entre las dos lecturas individuales.
- **Actualización perdida o reescritura de datos temporales** (Write - Write): ocurren cuando dos transacciones que intentan actualizar la misma fila son procesadas en un tiempo que permite que una de las actualizaciones sobrescriba a la otra.

Ejercicios

1. Para cada una de las siguientes ejecuciones, especifique si corresponde a Lecturas Sucias, Lecturas No repetibles o Actualización Perdida. Justifique su respuesta.

T1	T2	T3
R(a)	W(a)	
R(b)		
W(b)	W(c)	
R(a)		
		R(c)

Schedule 1

T1	T2	T3
R(a)	R(a)	
R(b)		
W(b)		
R(c)		
	W(a)	
		W(c)
		Abort

Schedule 2

T1	T2	T3
R(a)	R(a)	
W(b)		
W(a)		
R(c)		
	W(c)	W(c)

Schedule 3

2. Imagínese que está haciendo una transacción y se corta la luz. Indique que sucedería si a la base de datos no sigue las propiedades de **ACID** 🍋:

